

## **Pràctica de Sistemes Operatius**

**Curs 2025-2026**

### **The Citadel System**

| <b>Alumnes</b> | <b>Login</b>         | <b>Nom</b>                 |
|----------------|----------------------|----------------------------|
|                | <b>alba.rey</b>      | <b>Alba Rey Escobar</b>    |
|                | <b>berta.fornols</b> | <b>Berta Fornols Vaqué</b> |

# INDEX

|                                            |           |
|--------------------------------------------|-----------|
| <b>INDEX.....</b>                          | <b>1</b>  |
| <b>1. DISSENY.....</b>                     | <b>2</b>  |
| 1.1 FASE 1.....                            | 2         |
| 1.1.1 Diagrama de processos.....           | 3         |
| 1.1.2 Estructura de dades.....             | 4         |
| 1.1.3 Recursos del sistema utilitzats..... | 5         |
| 1.2 FASE 2.....                            | 5         |
| 1.2.1 Diagrama de processos.....           | 6         |
| 1.2.3 Recursos del sistema utilitzats..... | 8         |
| 1.3 FASE 3.....                            | 9         |
| 1.3.1 Diagrama de processos.....           | 9         |
| 1.3.2 Estructura de dades.....             | 10        |
| 1.3.3 Recursos del sistema utilitzats..... | 11        |
| 1.4 FASE 4.....                            | 12        |
| 1.4.1 Diagrama de processos.....           | 12        |
| 1.4.2 Estructura de dades.....             | 14        |
| 1.4.3 Recursos del sistema utilitzats..... | 15        |
| <b>2. PROBLEMES OBSERVATS.....</b>         | <b>17</b> |
| 2.1 FASE 1.....                            | 17        |
| 2.2 FASE 2.....                            | 17        |
| 2.3 FASE 3.....                            | 18        |
| 2.4 FASE 4.....                            | 19        |
| <b>3. ESTIMACIÓ TEMPORAL.....</b>          | <b>20</b> |
| 3.1 FASE 1.....                            | 20        |
| 3.2 FASE 2.....                            | 21        |
| 3.3 FASE 3.....                            | 22        |
| 3.4 FASE 4.....                            | 23        |
| <b>4.CONCLUSIONS I MILLORES.....</b>       | <b>25</b> |
| <b>5. NOMS DELS PROCESSOS.....</b>         | <b>26</b> |
| <b>6.BIBLIOGRAFIA.....</b>                 | <b>27</b> |

# 1.DISSENY

A continuació es presenta el disseny l'estructura implementada per desenvolupar la pràctica. S'inclouen els diagrames dels processos que s'han creat, així com les comunicacions entre ells. També les estructures de dades i els recursos utilitzats.

Com que la pràctica està dividida en 4 fases, s'ha vist oportú, dividir l'apartat en 4, així doncs, en cada cas s'explicarà el que s'ha utilitzat per a cada una de les fases.

## 1.1 FASE 1

La primera fase consisteix a desenvolupar l'estructura principal del maester, això inclou la lectura dels fitxers de configuració, dels stocks del regne i el corresponent emmagatzematge d'aquests paràmetres clau pel desenvolupament de la pràctica.

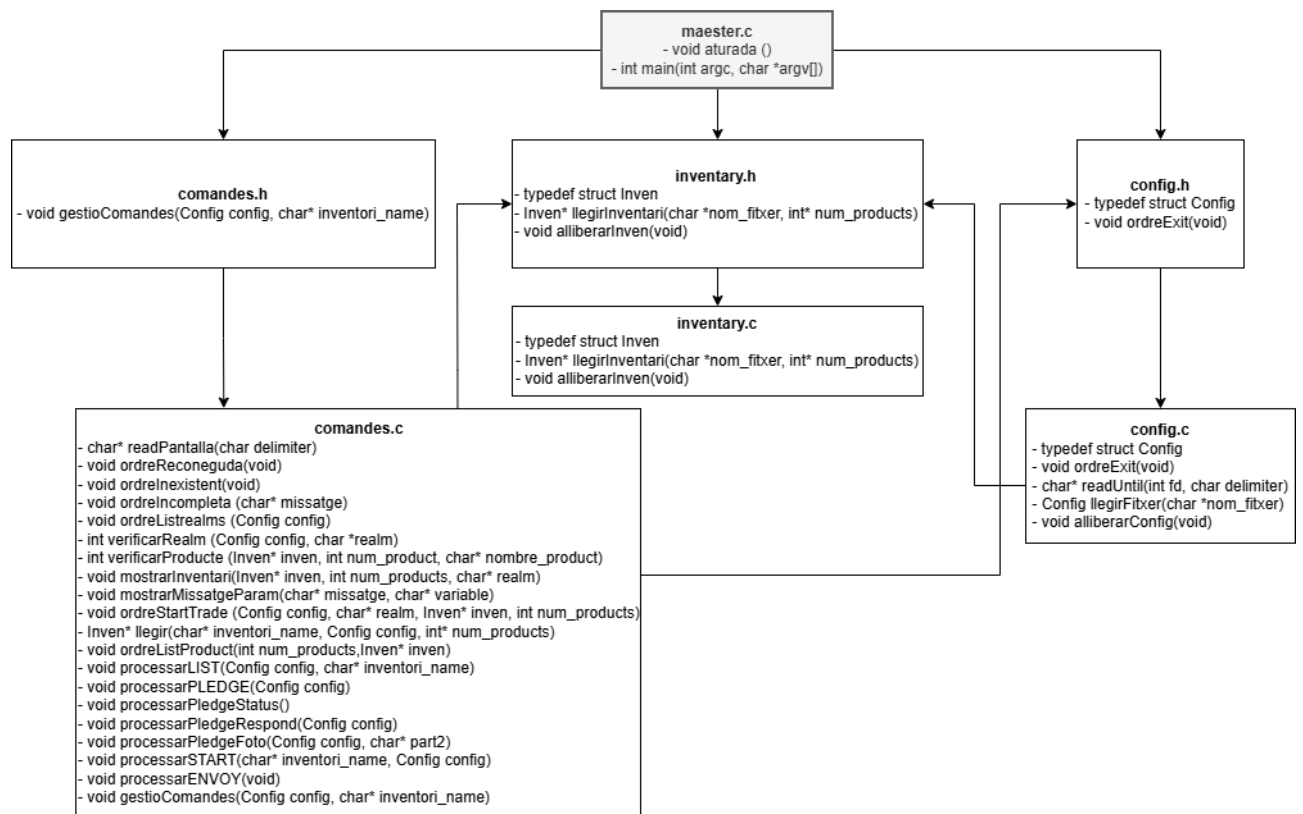
A més a més, el programa es basarà en una terminal de comandes, on en funció del que es vulgui realitzar, es cridarà a una execució o altra, això és el que s'ha preparat i començat a desenvolupar.

El menú consta de 9 possibles comandes, sigui per mostrar productes o regnes amb el qual el maester pot produir una comunicació, com iniciar peticions o respondre'n, també pot mostrar estat tant de les aliances com dels envois i pot començar intercanvis de productes.

En aquesta primera fase s'ha desenvolupat el menú complet, i tot i que es considera vàlida qualsevol de les comandes explicades anteriorment, només es produeixen la de llistar els regnes, llistar els productes, iniciar intercanvi de productes per afegir-los a la llista de la compra i la comanda de finalització del programa.

A continuació s'explicarà més detalladament l'estructura, processos i recursos que s'han utilitzat.

### 1.1.1 Diagrama de processos



Per la realització d'aquesta fase 1 de la pràctica, s'han creat els següents fitxers per modularitzar al màxim possible el codi i, d'aquesta manera poder gestionar d'una manera més ordenada la seva execució.

Primerament, el maester.c el que conté el main del programa i on es fa l'execució principal. Per tal de poder gestionar les comandes s'ha creat un mòdul amb comandes.h i comandes.c que únicament s'encarrega de rebre els paràmetres que escriu l'usuari per pantalla i fer el que calgui a cada comanda específicament.

A l'hora de voler escriure i llegir del fitxer de text, on es troben tots els productes de l'inventari del regne, s'ha creat el mòdul d'inventari.h i inventari.c. D'aquesta manera ens permet accedir d'una manera clara i neta

Per últim, el mòdul de config.h i config.c serveix per llegir i guardar la informació que es troba al fitxer de configuració del maester per poder fer-la servir fàcilment a altres mòduls i que estigui accessible.

Desde comandes.c es crida a inventari.h per poder tenir el contingut del fitxer inventari per gestionar les comandes. D'altra banda desde comandes.c també es crida a config.h per poder tenir les configuracions generals i poder realitzar correctament les comandes que necessiten aquesta informació.

A més a més, a config.c es fa servir funcions de inventari.h per poder fer l'alliberament degudament.

### 1.1.2 Estructura de dades

Pel que fa a les estructures de dades que s'han creat per aquesta primera fase ha estat dues estructures: una per gestionar la informació de l'inventari el qual el maester hagi d'accedir, i una altra per emmagatzemar la configuració inicial del regne corresponent, així com les seves pròpies aliances.

La primera estructura, Config, s'ha utilitzat per emmagatzemar totes les dades relatives al fitxer de configuració general del sistema. Aquesta estructura conté el nom del regne, el directori de la carpeta d'aquest mateix regne, el seu IP i port, a més a més, dels envois associats, és a dir, els regnes amb els quals es poden establir connexions, les adreces IP i els ports corresponents i el nombre de connexions que pot establir el regne.

Aquesta estructura s'omplirà inicialment just en començar a executar el programa, perquè el maester pugui saber a quin regne correspon i amb quins pot establir connexions.

A continuació es deixa de manera visual l'estructura corresponent.

```
typedef struct{
    char *nom_regne;
    char *directori_carpeta;
    char *envois_regne;
    char *ip_regne;
    char *port_regne;
    char **regnes_connexions;
    char **ip_connexions;
    int *ports_connexions;
    int num_connexions;
}Config;
```

La segona estructura, Inven, serveix per llegir el fitxer stock en binari del regne i guardar tots els productes, amb el seu nom, la quantitat i el pes. D'aquesta manera sempre que es vulgui accedir els productes guardats, tindrem una estructura més fàcil de gestionar i treballar amb les dades, en el moment de mostrar-les per pantalla o modificar-les.

Aquesta estructura es llegirà quan es vulgui treballar amb productes, és a dir, quan es vulgui iniciar intercanvi amb altres aliances o quan simplement es vulguin mostrar els productes amb els seus paràmetres corresponents.

A continuació es deixa de manera visual l'estructura corresponent.

```
typedef struct{
    char nom[100];
    int quantitat;
    float pes;
}Inven;
```

### 1.1.3 Recursos del sistema utilitzats

Pel que fa als recursos que s'han utilitzat en aquesta pràctica, dels que s'han vist i treballat a classe, han estat pocs, ja que la primera fase era una base per poder començar a modular el codi en posteriors fases. Tot i això, s'han fet servir, file descriptors i signals. A més a més, de treballar amb memòria dinàmica i punters.

Tot seguit s'explicarà la funcionalitat i la implementació de cada un dels recursos.

En primer lloc, els file descriptors, aquests es fan servir per llegir, escriure i gestionar fitxers, és a dir, obrir o tancar fitxers, així com crear-los i poder llegir-los i editar-los. A més a més, de poder escriure i llegir per pantalla.

S'han utilitzat open i close, per poder obrir els fitxers de text o binaris, en aquest cas, el config.dat i el stock.db. Addicionalment, de crear i escriure i/o llegir el fitxer de la llista de la compra necessitat en realitzar intercanvis de productes.

Per gestionar l'aturada del programa, a part de la comanda exit, la qual acaba l'execució del programa. També s'ha de controlar el senyal SIGINT, aquest fa referència al Cntrl+C, per assegurar l'aturada controlada del Maester. Quan l'usuari interromp el programa, la funció aturada s'executa, tancant els recursos corresponents i alliberant la memòria.

Ja per acabar, cal destacar que s'ha utilitzat memòria dinàmica per aprofitar el màxim la memòria i no perdre espai. En aquest cas, doncs, l'estructura del stock és dinàmic, ja que no especifica quants productes pot tenir. També s'ha implementat memòria dinàmica, en el cas de treballar amb variables locals.

## 1.2 FASE 2

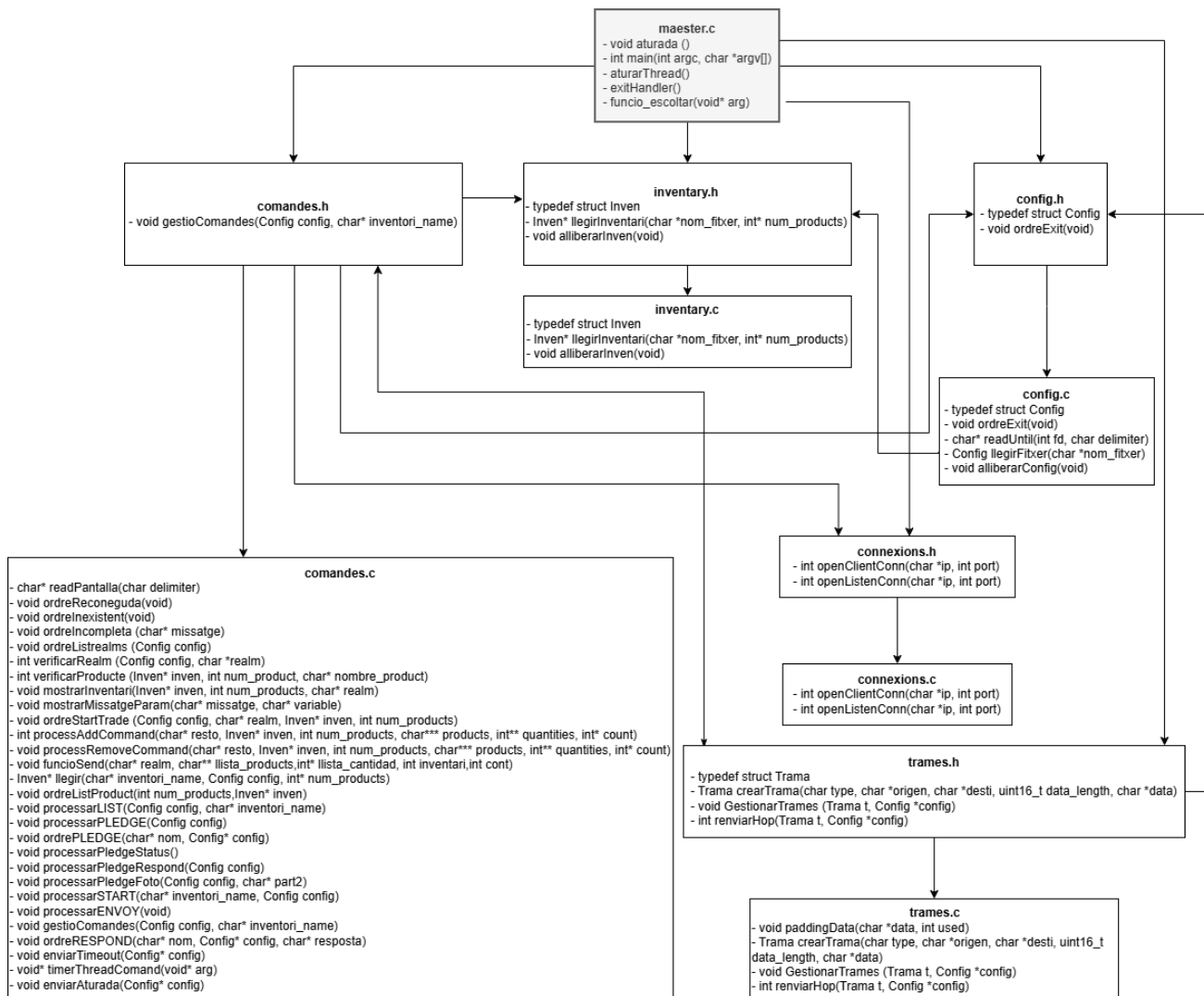
En aquesta segona fase es desenvolupa tota la comunicació entre regnes. Cada procés maester passa a actuar com un node d'una xarxa distribuïda capaç de rebre, enviar i encaminar missatges entre diferents màquines mitjançant sockets.

Quan un maester rep una trama, comprova si és el destinatari final o si ha d'actuar com a hop, en tots casos, farem comprovacions per mirar que la trama sigui correcta.

A més a més, peremt iniciar missatges de comerç, però el processat es treballarà a la següent fase.

## 1.2.1 Diagrama de processos

Igual que la fase anterior, hem generat un esquemàtic dels fitxers creats i com es relacionen entre ells, per a poder tenir una idea general de com s'està desenvolupant el sistema.



Com es pot veure s'han afegit 4 fitxers nous per modularitzar el programa de la manera més òptima i clara, de tal manera de que sigui escalable per a les següents fases restants.

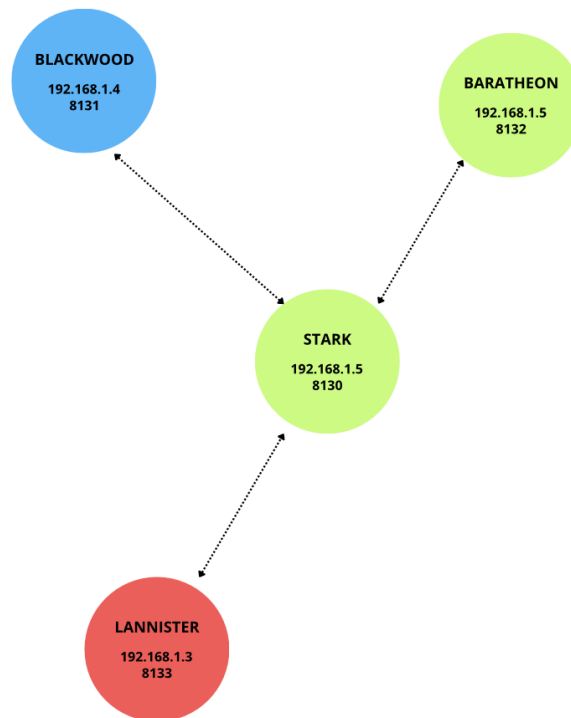
Els fitxer connexions. h i connexions.c estan enfocats a gestionar el funcionament base dels sockets, tant per deixar-lo escoltant com per obrir la connexió amb el client.

Els fitxers trames.h i trames.c gestionen la creació, el reciviment i el reenviament de les trames, bàsicament s'encarreguen de la informació que circula a través del protocol de comunicació entre els diferents maesters.

A més a més, en aquesta fase ja hem introduït la comunicació entre diferents processos, fet que implica establir connexions entre diversos regnes del sistema. Per aquest motiu hem elaborat un esquema funcional de les diferents xarxes i regnes, amb l'objectiu d'identificar en quina màquina virtual es troba cadascun, així com la seva IP i el port de connexió (tal com s'especifica dins de cada fitxer de configuració del regne).

També s'han tingut en compte tant les connexions directes com els salts intermedis (*hops*) necessaris, així com la ruta per defecte.

Així doncs la nostra arquitectura de processos consta de 4 regnes repartits entre Puigpedrós (de color verd), Matagalls (de color vermell) i Montserrat (de color blau) amb les seves corresponents IP i port de connexió.



### 1.2.2 Estructura de dades

A la primera fase vam crear dues estructures de dades bàsiques per poder guardar informacions.

En aquesta segona fase, per a realitzar la implementació i poder estar enviant i actualitzant les estructures mitjançant punters, ha set necessari ampliar l'estructura de dades de configuració.

```
typedef struct{
    char *nom_regne;
    char *directori_carpeta;
    char *envoys_regne;
    char *ip_regne;
    char *port_regne;
    char **regnes_connexions;
    char **ip_connexions;
    int *ports_connexions;
    int num_connexions;
    char** connexions_demanades;
    int num_demanades;
    int esperant_resposta;
    char* nom_esperat;
```



```
}Config;
```

En aquest cas doncs, necessitem poder guardar amb quines regnes estar realitzant aliances i de les quals podem deixar que ens facin altres tipus de sol·licituds. A més a més, necessitem saber si hem rebut resposta d'acceptació o no i per assegurar-nos de que rebem de qui volem rebre necessitem també poder controlar-lo.

Sabem que encara estem en procés de desenvolupament d'aquesta fase i de la pràctica globalment, per tant també entenem que segurament pot encara anar patint modificacions.

Com ja hem comentat, en aquesta fase quan realitzes comunicacions i enviaes informació, ho fem mitjançant unes trames, les quals tenen uns formats específics, amb més d'un paràmetre. Per poder gestionar-ho hem creat una nova estructura de dades.

```
typedef struct{
    char type;
    char origin[20];
    char destination[20];
    uint16_t data_length;
    char data[275];
    uint16_t checksum;
}Trama;
```

Per poder fer l'enviament i el rebiment de les dades de les trames d'un sol read i write, s'ha creat una estructura únicament per aquesta funció. Cada component de la struct s'ha declarat de manera que compleixi amb els paràmetres i número de bytes establerts per a cadascun. A cada trama varien els valors que prenen les variables que la conformen.

Aquesta estructura es la informació que s'intercanvia en tots els processos de comunicació entre processos. Es la informació que circula pels sockets, és a dir, la que passa per les diferents màquines. D'aquesta manera s'estableix un paquet fixe d'enviament i recepció que permet generalitzar la comunicació.

### 1.2.3 Recursos del sistema utilitzats

En aquesta segona fase, a diferència de l'anterior on funcionava de manera seqüencial i només havia de realitzar una acció, ara necessitem que estigui constantment connectada per estar pendent si rep trames, a més a més, de continuar amb les funcionalitats de la fase anterior, és a dir, una aplicació concurrent. Degut això hem necessitat utilitzar threads perquè el maester pugui executar múltiples tasques simultàniament sense bloquejar el bucle principal de comandes i sockets per establir connexions.

El maester fa servir un fil dedicat (funcio\_escoltar) que executa la funció accept dels sockets, un altre recurs que s'explicarà a continuació, permeten rebre les connexions d'altres regnes sense interrompre el bucle main que espera comandes de l'usuari.

També s'ha implementat un fil temporal per gestionar el temps d'espera de resposta d'una sol·licitud d'aliança. Així doncs, contarà dos minuts i en funció de si s'ha rebut o no la trama de contesta, enviarà una trama de timeout per indicar error en cas de superar el temps límit d'espera.

El segon recurs implementat en aquesta fase, són els sockets. Per poder fer l'intercanvi d'informació entre Maesters, hem necessitat utilitzar els sockets per mantenir connexions entre ells, tot sabent la seva IP i port.

Cada vegada que volem realitzar una acció nova de les comandes on cal establir comunicació, obrim una comunicació entre els dos maesters, incloent-hi els hops intermediaris per les comunicacions que no siguin directament connectades. Un cop enviem la trama i el maester destí la rep i la processa, enviarem la resposta ack o nack per indicar que l'hem rebuda i acabarem la comunicació fins que no volem fer una nova comanda.

A més a més, el que estarà escoltant permanentment si rebem trames és el thread, el qual sempre té la comunicació oberta per tothom que li vulgui demanar o enviar informació.

## 1.3 FASE 3

La fase anterior es basava a realitzar les comunicacions perquè poguessin comunicar-se entre regnes, amb la finalitat de poder fer intercanvi de béns mitjançant fitxers comercials. Així doncs, aquesta fase es basa en la implementació d'aquestes comandes que permetran realitzar intercanvis i comercialitzar entre regnes.

Per fer-ho a més a més, hi haurà l'intercanvi de segells per a establir comunicacions i assegurar-se de qui ho està enviant, cosa que no fèiem a la fase anterior.

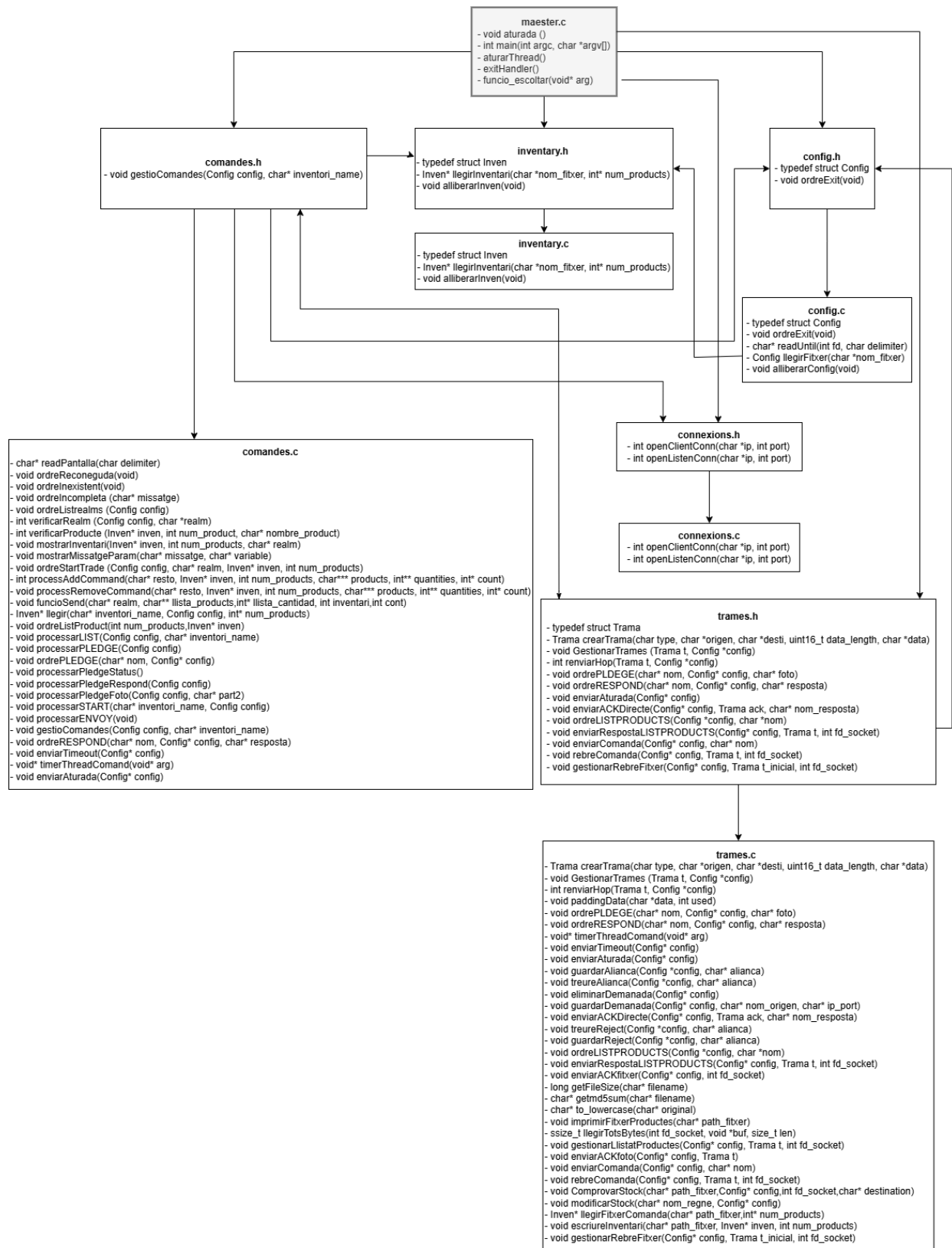
Amb aquesta fase, acabarem d'implementar totes les funcionalitats que pot dur a terme el sistema a través de les comandes.

Tot i això, encara només es pot produir una comunicació alhora, ja que encara no hem implementat els envois, que es duran a terme més endavant.

### 1.3.1 Diagrama de processos

A la fase anterior ja vam introduir la comunicació entre diferents processos, fet que implicava establir connexions entre diversos regnes del sistema. Per aquest motiu es va elaborar un esquema funcional de les diferents xarxes i regnes. Per a fer aquesta fase, no s'han dut a terme modificacions pel que fa a la implementació de la xarxa.

Al diagrama següent es mostren les funcions públiques, que accedeixen altres mòduls, als fitxers .h i les funcions privades, que només es criden dins dels .c del mateix mòdul.



### 1.3.2 Estructura de dades

Pel que fa a les estructures de dades, en aquesta fase no s'ha necessitat la implementació de cap nova estructura. Simplement, hem augmentat l'estructura config, ja que necessitem

controlar més paràmetres i és l'estructura base que ens ajuda a poder tenir-ho tot controlat de cada regne.

```
typedef struct{
    char *nom_regne;
    char *directori_carpeta;
    char *envoys_regne;
    char *ip_regne;
    char *port_regne;
    char **regnes_connexions;
    char **ip_connexions;
    int *ports_connexions;
    int num_connexions;
    char** connexions_demanades;
    char** ip_port_demanades;
    int num_demanades;
    int num_reject;
    char** reject;
    char** aliances;
    int* permis_trade;
    int num_aliances;
    int esperant_resposta;
    char* nom_esperat;
    char* foto_esperada_path;
    long foto_mida_total;
    char* foto_esperada_md5;
}Config
```

Els paràmetres afegits han estat una de ip\_port, per a quan ens sol·liciten connexions per saber l'ip i el port d'origen per si s'ha de convertir en una aliança directa. Pel que fa al pledge status, per saber l'estat de tots els regnes, tenim demanades, per saber les que estant pendents de resposta, les reject que són sol·licituds rebutjades i les aliances, que són les que han estat satisfactòriament connectades. A més a més, tenim una nova variable anomenada permis\_trade perquè és necessari, haver fet un list products abans de poder iniciar qualsevol intercanvi, aquesta serà la variable que ens indicarà si s'ha realitzat o no i si té permís per poder començar a intercanviar productes. L'últim canvi significatiu, ha estat la incorporació de variables, per a poder gestionar l'enviament del segell, quan es vol demanar una aliança.

A més a més, en aquesta fase s'ha treballat molt amb l'estructura de l'inventari per poder fer l'intercanvi de productes entre regnes, però continua tenint els tres paràmetres bàsics; nom, preu i pes.

### 1.3.3 Recursos del sistema utilitzats

Per aquesta fase no hem utilitzat nous recursos dels quals hem vist a classe. Continuem fent servir els que ja hem esmentat fins ara a les anteriors pràctiques.

Segurament de cara a la següent fase, per poder realitzar els envois, faran falta altres recursos.

A més a més, tenim en compte que hem d'implementar mutex per poder evitar que múltiples processos necessitin escriure per pantalla a la vegada. Cosa que tenim pensat implementar-ho de cara a l'entrega final.

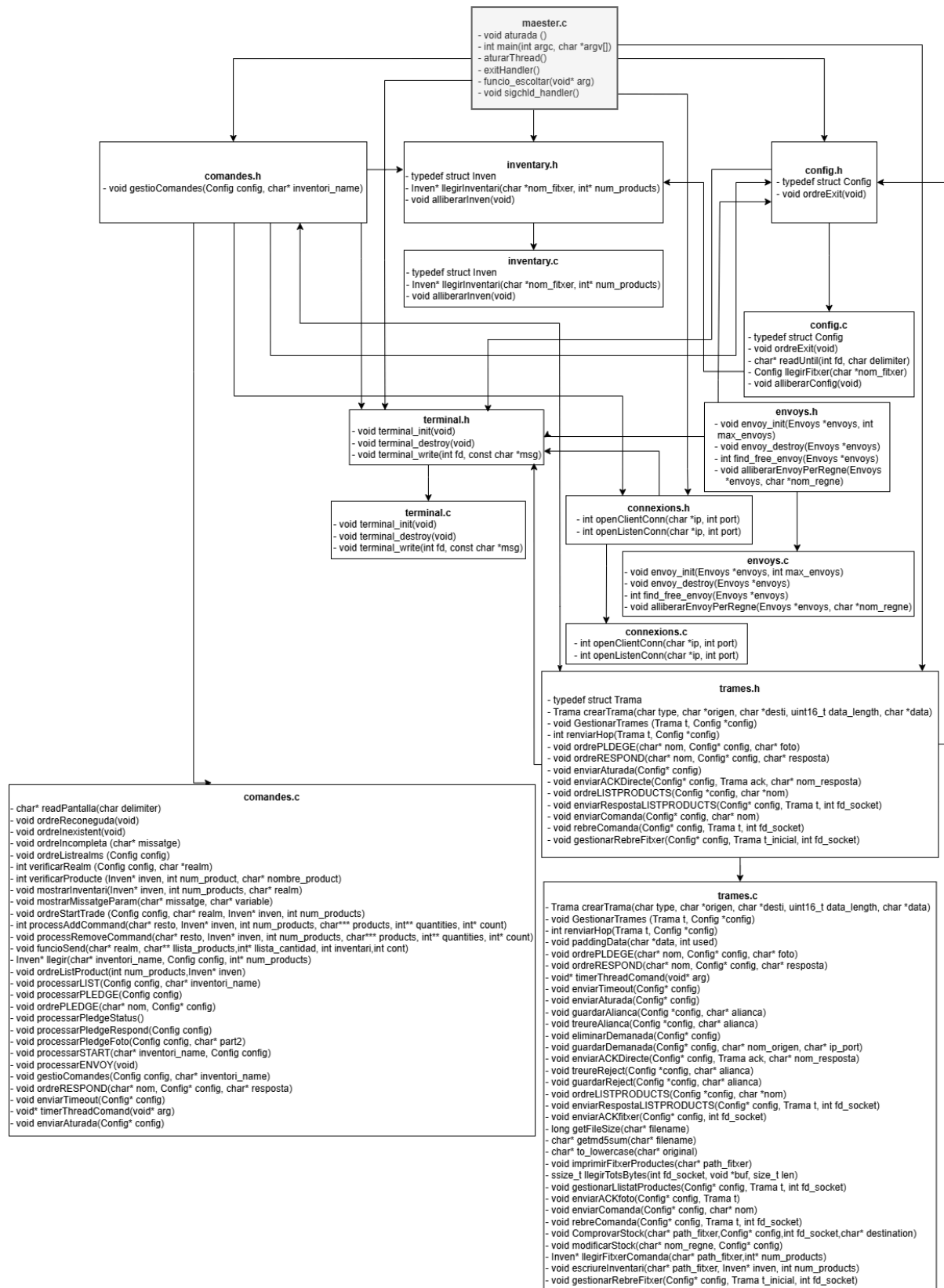
## 1.4 FASE 4

El que hem vist al llarg de les fases, és que s'ha pogut produir inicialment un menú per a gestionar els maesters, unes lectures de fitxers i finalment un intercanvi de productes entre comerços, que inclou la comunicació entre processos.

Aquesta última fase, doncs, consistia en poder implementar uns ajudants, del maester i així poder evitar només poder produir una comanda o comunicació amb un sol maester. Els envoys, podran ajudar al maester a rebre i enviar fitxers entre maesters, tot i que aquest sigui l'encarregat principal de gestionar-ho.

### 1.4.1 Diagrama de processos

Igual que en altres fases, s'ha generat un esquemàtic per poder veure els fitxers que conformen el codi i com es relacionen. D'aquest manera es pot tenir una visió general del desenvolupament final de la pràctica. Com es pot observar, hi ha hagut canvis respecte als fitxers definits a fases prèvies, s'han afegit dos fitxers nou per la gestió dels envoys, i s'han eliminat dos degut a que no aportaven organització al codi i tenien molt poc ús. De tal forma, la imatge següent mostra com queda l'esquemàtic final.

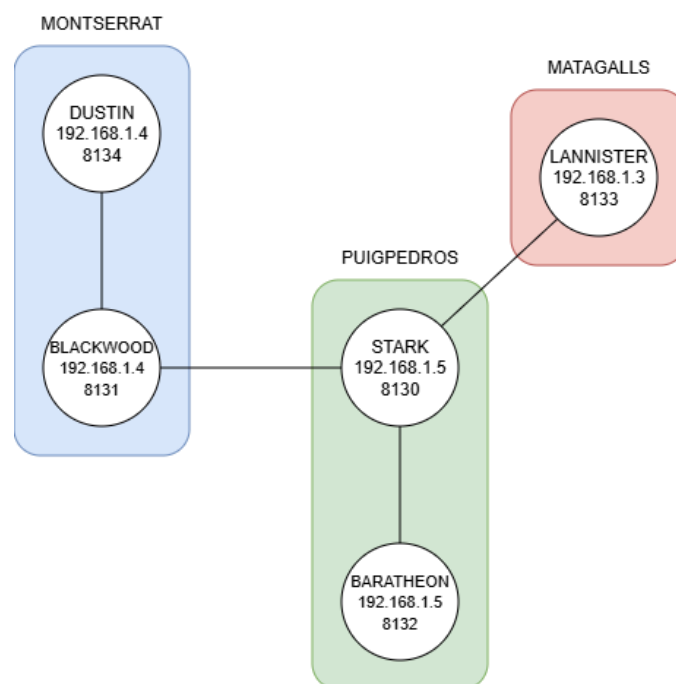


Com es pot veure, els fitxers envoy.c i envoy.h son els fitxers afegits per gestionar la creació, els recursos restants i l'alliberament de la seva memòria utilitzada.

A més a més, s'han afegit funcions al fitxer comandes per poder gestionar l'ús d'envoys en cada cas. Definir com es comporten i com es comuniquen amb el procés pare per que aquest pugui comunicar-se amb la resta de regnes. Per tant, fent forks es poden crear subprocesos on s'executen els envoys i així el pare poder gestionar altres coses a l'hora.

Com s'havia mencionat a apartats de fases anteriors, únicament s'havien tingut en compte 4 realms per la realització de la pràctica. El fet de tenir només 4 realms configurats a l'arquitectura de processos afectava significativament al correcte funcionament del codi, ja que en cas d'haver de fer hops només era possible realitzar un per la quantitat de realms i la seva disposició de connexions intermèdies. A la correcció de la pràctica anterior es va recalcar aquest punt i s'ha afegit un realm nou per poder assolir totes les funcionalitats demanades i, per tant, poder fer dos hops per arribar al destí.

Per tant, el diagrama de connexions establertes entre els realms queda de la següent manera després de realitzar els canvis pertinents:



## 1.4.2 Estructura de dades

Pel que respecta a les estructura de dades Config creada anteriorment, s'ha modificat traient variables definides dins que no eren estrictament necessàries que fossin globals. D'aquesta manera, s'ha optimitzat els recursos utilitzats reduint la càrrega de memòria de l'estructura Config.

```
typedef struct{
    char *nom_regne;
    char *directori_carpeta;
    int envoys_regne;
    char *ip_regne;
    char *port_regne;
    char **regnes_connexions;
    char **ip_connexions;
    int *ports_connexions;
```

```

int num_connexions;
char** connexions_demanades;
char** ip_port_demanades;
int num_demanades;
int num_reject;
char** reject;
char** aliances;
int* estat_aliances;
int* permis_trade;
int num_aliances;
int esperant_resposta;
char** nom_esperat;
time_t* time_recepcio;
}Config;

```

Hem implementat una nova estructura de dades, per gestionar els envoyos, així tenir controlat en tot moment, cada envoy, amb el seu estat i missió corresponent.

```

typedef struct{
    int num_envoys;
    int* status;
    char** mission;
    pid_t *pids;
    int* pipe_read_fd;
    char** waiting_realm;
    pthread_mutex_t mutex;
}Envoys;

```

Tenim una variable per saber el nombre d'envoys possibles que tenim, una per saber l'estat d'aquests envoys (0 = lliure i 1 = ocupat). Un string per indicar quina missió està executant, així si volem veure els estats sabrem amb qui està ocupat i fent què. Després tenim guardat el pid del procés fill del fork, el descriptor per saber per on llegir les pipes i en aquest cas hem afegit també una variable per saber de quin realm estàvem esperant resposta pel tema del pledge. A més a més, del mutex, que esmentarem a continuació.

### 1.4.3 Recursos del sistema utilitzats

En aquesta última fase, com ja s'ha comentat el principal objectiu era poder incloure els envoyos, els quals fessin part de la feina del maester i així controlar diferents operacions simultàniament.

Per a poder assegurar, la concurrència, comunicació i sincronització s'han utilitzat forks, per crear fills independents que executen les missions en les quals es necessita comunicació entre regnes, és a dir, list products realm, pledge i start trade realm. Cada envoy, és un d'aquests processos que hereta l'estat inicial i executa la funció específica en segon pla, un cop s'acaba avisa al pare de la seva finalització i així tornem a tenir un envoy disponible. Per poder fer aquesta comunicació entre pare i fill, s'han utilitzat pipes unidireccionals, on el fill escriu el resultat de la missió i el pare ho llegeix, tot mitjançant el SIGCHLD. Aquesta pipe es crea abans del fork i es tanca correctament en ambdós processos per evitar pèrdues.



Com que tenim una estructura de dades envois, per controlar els estats de cadascú i el que estan executant en tot moment, hem fet servir mutex que protegeix les operacions quan volem canviar l'estat d'un envoy. A més a més, també hem utilitzat mutex per tot allò que es vol mostrar per pantalla, així doncs evitem conflictes entre temps d'arribada i evitem que més d'un entri en zona crítica a la vegada.

## 2.PROBLEMES OBSERVATS

### 2.1 FASE 1

Durant la realització d'aquesta primera fase es van trobar més problemes simples de solucionar a causa de la menor dificultat. Majoritàriament, eren problemes de no recordar de com fer certes coses bàsiques que calia refrescar i entendre el concepte general de la pràctica.

Un dels problemes va ser haver fet la funció de la comanda start trade massa gran, de més de 60 línies, que excedia per molt el límit que indica a la guia d'estil. Per solucionar-ho va fer falta subdividir aquesta funció per fer totes les accions que es duïen a terme dins de l'execució de la funció global però en petits trossos de codi.

El problema que s'havia trobat en intentar fer això era que calia passar-li i retorna moltes variables no globals i que calia mantenir per següents execucions. D'aquesta manera, passant els paràmetres per referència a les subfuncions es va poder distribuir el pes del codi de la funció original en altres que no han de retornar diversos paràmetres perquè estan referenciats amb punters.

També va haver-hi problemes a l'hora de fer les comprovacions finals, ja que al document explicatiu de la pràctica hi ha certs aspectes i condicions bastant específiques per l'execució que no es mencionen. Per tant, en estar fent les comprovacions amb el document que dona exemples de possibles execucions ens vam adonar que hi havia comandes que no estaven fetes o altres coses més específiques que també faltaven per fer.

Gràcies al fet que es va acabar d'hora i les comprovacions es van fer dies abans de la data límit es van poder resoldre i corregir aquests aspectes incorrectes.

Finalment, una de les primeres dificultats que es va afrontar va ser el fet de tenir compartit tot el màxim a temps real possible i no intercedir als canvis de l'altra companya. De tal manera, vam organitzar tot perfectament per evitar petits problemes en ajuntar el codi realitzat individualment i que es pugui treballar a l'hora, i així, aprofitar al màxim el temps.

### 2.2 FASE 2

En realitzar aquesta segona fase de la pràctica, s'han patit diversos problemes que afectaven tant l'execució del programa com a la gestió de memòria. A diferència de la primera fase, aquests han sigut molt més difícils de detectar i arreglar.

En primer lloc, entendre el funcionament del programa va ser complicat degut a totes les trames que s'han de gestionar el seu enviament i entendre en què casos enviar cadascuna. Lligat amb el tema de les trames, hi ha un problema a l'hora de rebre la trama a través d'un hop perquè només es rep la IP i el port de l'origen, però si no se sap de qui prové no es pot relacionar el nom amb la IP i el port coneguts mitjançant la trama.

Un dels principals problemes pel desenvolupament d'aquesta fase ha sigut el temps. Actualment, hi ha diverses entregues per setmana de la resta d'assignatures que tenim i no

s'ha pogut gestionar per poder entregar la fase amb totes les funcionalitats esmentades a l'enunciat.

D'altra banda, va haver-hi problemes a l'hora d'estructurar el codi, al començament es va ficar codi de sockets molt barrejat, sense saber on s'executaria aquella línia, en quina terminal. Un cop va anant avançant el codi, ens vam adonar que no es podia sostenir d'aquesta manera i de cara a les fases 3 i 4 seria pitjor. Es va millorar estructuralment del codi modularitzant molt més i intentant que sigui el més clar possible per poder ampliar-ho més fàcilment en fases futures.

També es van detectar problemes a l'hora de fer la comanda èxit, ja que no terminava l'execució. Això era a causa que es feia servir la funció `pthread_join()` que esperava que tots els fils acabin, en canviar per la funció `pthread_detach()` fa que els fils acabin obligatòriament. En fer servir Threads per fer l'espera en segon pla dels programes feia que mai acabessin perquè es quedava bloquejat esperant alguna trama i no continuava l'execució del codi, per tant, no detectava que el codi estava indicant amb una flag que ha de finalitzar el Threads.

Finalment, el problema que més ha endarrerit l'entrega de la pràctica ha sigut l'enviament de la trama NACK a través d'un hop perquè aquesta no conté ni camp d'origen, destí ni dades no proporciona cap informació a l'hop de cap a on s'ha d'enviar aquesta trama. Per tant, aquesta comunicació únicament funciona en el cas que es faci sense hop intermedi.

## 2.3 FASE 3

Durant el desenvolupament d'aquesta tercera fase de la pràctica, s'ha notat com la complexitat ha augmentat considerablement en molts aspectes, ja que s'han de gestionar moltes coses i cada vegada es complica més l'estructura general del codi per fer que quedi el més endreçat i comprensible. Els problemes principals han estat a l'hora de l'enviament de fitxers en diferents trames i la gestió de memòria.

Un dels problemes més crítics va ser l'enviament de la imatge del regne en fer un pledge de petició d'aliança. Al començament va costar de plantejar l'enviament repartit en diferents paquets, ja que no es podia fer en una única trama. El problema era que s'estava tractant cada trama de manera independent i no com a un conjunt. Una vegada es va solucionar aquest, es va notar que en fer-ho amb hops no funcionava perquè el programa estava obrint i tancant la comunicació tota l'estona en comptes de deixar-la oberta per tal que els paquets d'un mateix fitxer puguin arribar tots. Es va aconseguir fer tenint en compte la mida de les dades que s'envia a la capçalera d'enviament i tenint un comptatge de la mida de les dades rebudes.

Pel que fa a gestió de memòria, es van haver d'arreglar diversos errors de double free que sortien a l'hora de comprovar els errors amb el valgrint. També, fent ús de l'eina valgrint va sorgir que quan s'acabava el programa fent un exit o un `ctrlC` hi havia falles per culpa de la finalització del thread. El problema és que el thread té la funció d'esperar i, per tant, en fer ús de la funció `read` es bloqueja l'execució del thread en aquell punt, per aquest motiu fent un `pthread_join` no s'acabava mai el programa perquè es quedava esperant que el thread acabés per si sol. Per solucionar-ho, es va utilitzar la funció `pthread_cancel` que atura

l'execució del thread, d'aquesta manera es va poder eliminar l'error que s'indicava al valgrint.

A més a més, el temps, el treball endarrerit i la càrrega de treball actual ha sigut un dels problemes principals. La fase 2 no es va poder finalitzar completament, de fet, van quedar moltes coses pendents de resoldre també per falta de temps en aquell moment. D'aquesta manera es van implementar directament a aquesta fase algunes de les funcionalitats que a la fase 2 es preparaven i organitzaven per després acabar-les a la fase 3. Això ha comportat haver de fer més feina ara. D'altra banda, actualment el curs acadèmic implica molta càrrega de treball perquè a totes les assignatures s'han d'entregar pràctiques.

## 2.4 FASE 4

En aquesta última fase de la pràctica, s'han hagut de resoldre tots els problemes pendents, per tant, s'han hagut d'enfrontar molts més reptes per assegurar que la pràctica no tingui cap error o condició incompleta en la seva totalitat.

El primer obstacle va ser l'organització del temps i la càrrega de treball actualment al curs. El fet d'haver de finalitzar la pràctica, estudiar pels exàmens i preparar l'entrevista ha suposat moltes hores de treball durant aquestes vacances de nadal.

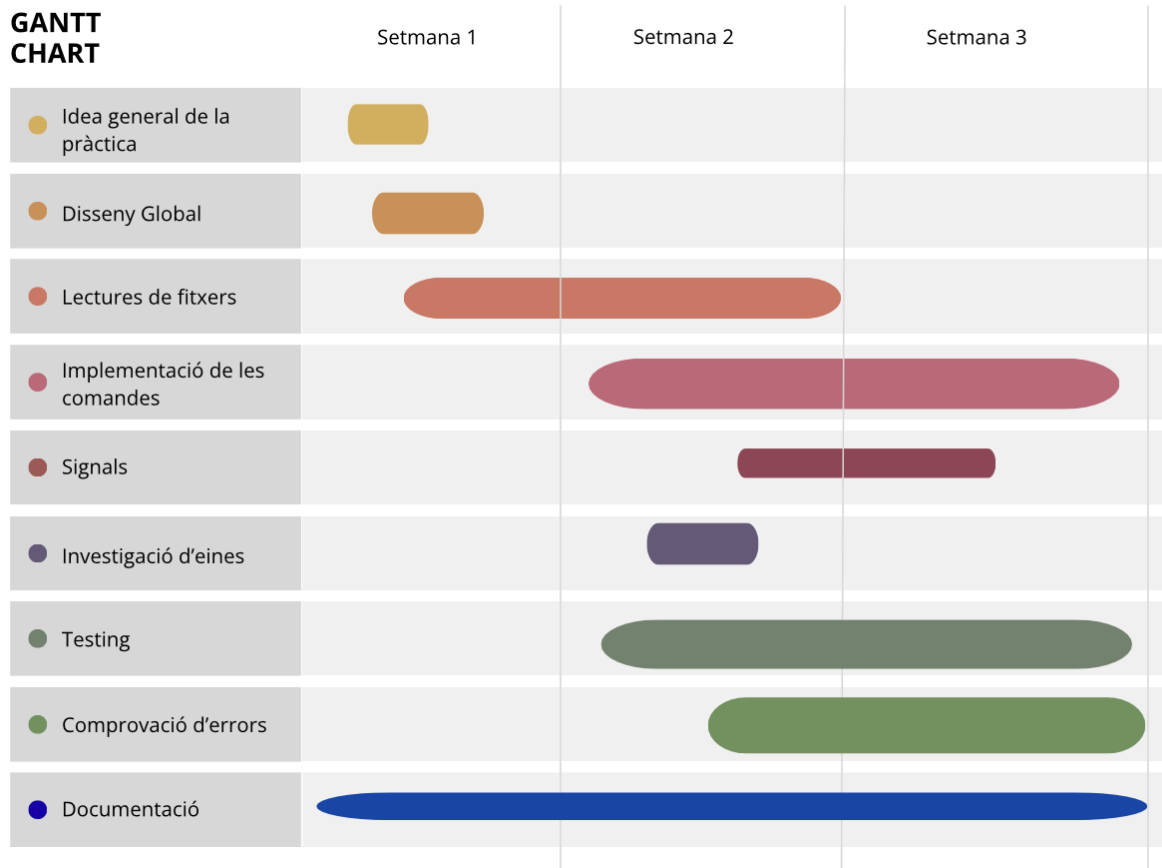
Un problema relacionat amb el codi, va ser canviar la manera de gestionar el timeout de la trama pledge. En un inici s'havia fet amb un thread adicional que esperes 120 segons i quan acabes avises. S'ens va indicar que seria millor gestiona-ho d'un altre manera més eficient, que en aquest cas, es va optar per guardar l'hora d'arribada de la primera comanda i l'hora d'arribada de la resposta. D'aquesta manera no s'executa res més a l'hora i es fes eficient. A més aquesta funcionalitat s'estava implementat malament. Abans, una vegada s'acabava el temps de timeout sortia un missatge, de manera que no es tenia en compte l'arribada de la resposta. En canvi, ara únicament surt el missatge de que s'ha superat el timeout quan arriba la resposta i es comprova que estigui dins del temps.

A l'hora de fer la implementació dels envois no va haver cap problema greu, excepte en la funcionalitat de start trade. Aquest envoy va donar bastants problemes perquè al enviar del procés fill al procés pare no es feia bé la transmissió de dades. Això era perquè el fill estava finalitzant abans que el pare pogues guardar les dades rebudes. El problema resultant era que no s'actualitzava mai l'stock del realm que havia fet l'start trade.

Per últim, al començament de la fase hi havia molts dubtes sobre el feedback donat de la fase anterior, l'enunciar de la fase actual, el funcionament del nostre codi, etc. Gràcies als becaris es van poder resoldre la gran majoria d'aquests i es va poder entregar intentant complint al màxim tots els requisits.

## 3. ESTIMACIÓ TEMPORAL

### 3.1 FASE 1



Pel que fa a la fase 1, la vam dur a terme amb 3 setmanes.

La primera setmana va ser de lectura i visualització de la pràctica generalment, a més a més de dissenyar la fase 1 per a ser validada.

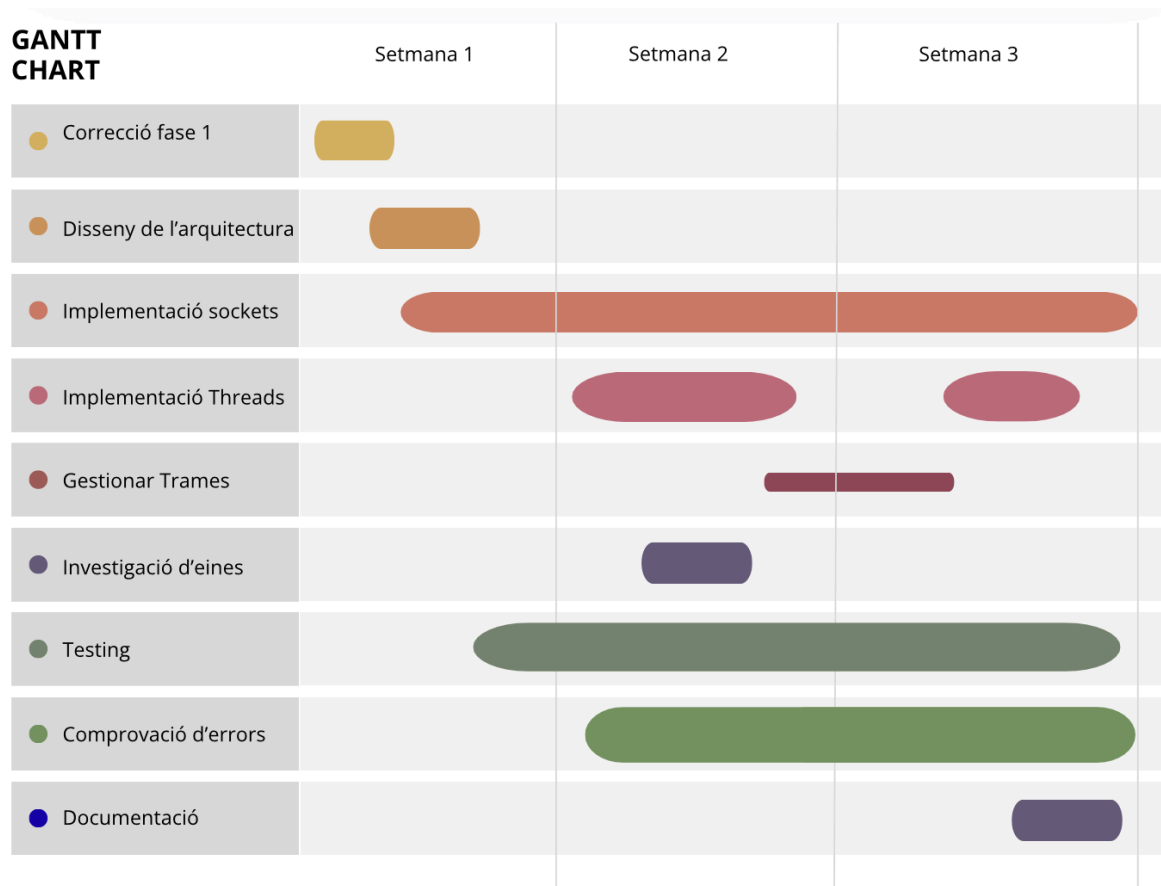
La fase de codificació, la vam anar desenvolupant al llarg de les 3 setmanes, centrant-nos inicialment amb les lectures de fitxers, seguit de les comandes del menú. I un cop vam treballar els signals a classe, vam implementar-ho a la fase, tot i que ens va costar una mica.

Pel que fa a investigació d'eines, es van dur a terme la cerca, per trobar eines que ens ajudessin a analitzar i separar les comandes (a l'apartat de bibliografia es veuen les eines buscades).

La part que més ens ha portat temps, i a la que vam dedicar-hi més esforç, ha estat a la part de testear i detectar errors, ja que cada comanda pot comportar diversos errors diferents, a més a més d'altres genèrics que poden tenir lloc.

Ja per acabar, la memòria, va estar elaborada paral·lelament al disseny, per facilitar el desenvolupament.

## 3.2 FASE 2



Primerament, abans de començar amb els requisits de la fase 2, es van arreglar tots els errors esmentats al feedback de l'entrega anterior. Fent això, es va poder aconseguir una base molt més sòlida i fiable amb la que treballar per poder aconseguir un resultat millor.

Un cop teniem plantejat com fer aquesta nova fase, es va dibuixar com seria la xarxa de maesters a implementar, d'aquesta manera podríem començar a crear els fitxers de rutes amb les connectades i els defaults. Amb això, ja hi havia una base amb la que començar a treballar amb les comunicacions.

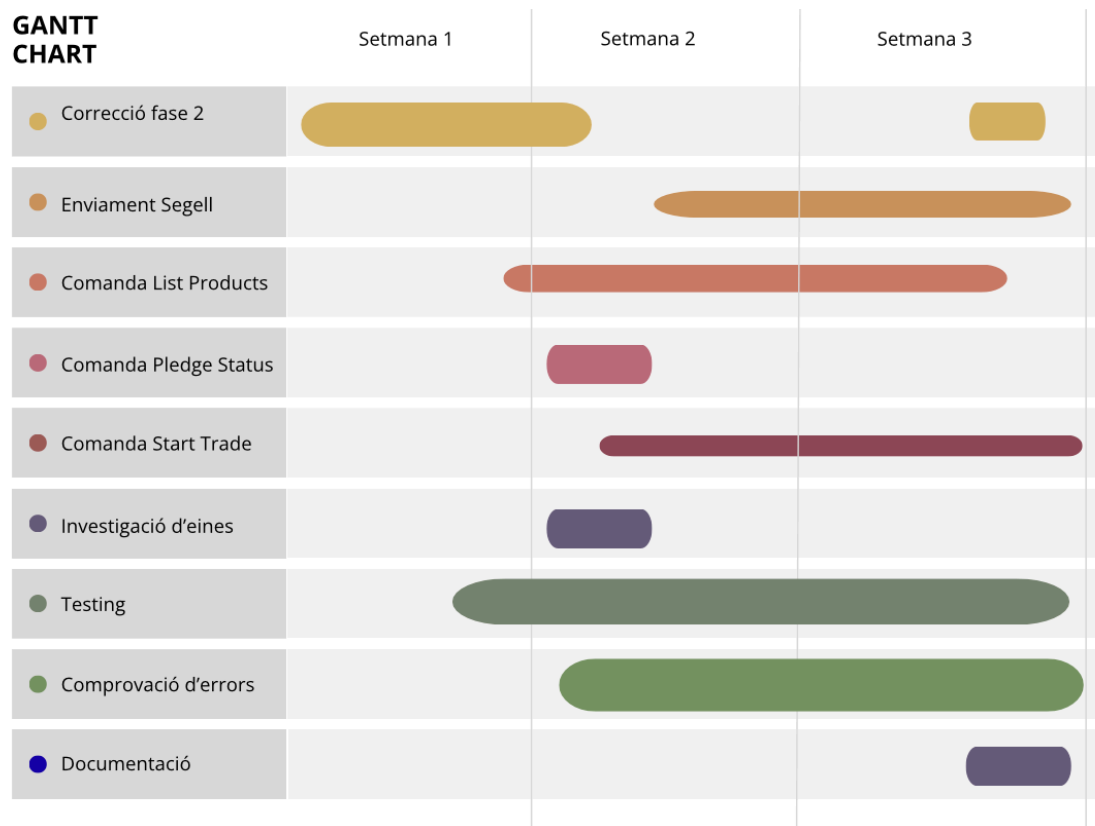
Amb la implementació dels sockets es amb els que més s'ha invertit temps degut a diversos problemes trobats i la dificultat de gestionar-los. D'altra banda, es van afegir threads per poder gestionar les trames rebudes i l'espera de temps de dos minuts.

La gestió de trames no ha ocupat molt pero no està acabada ja que no s'entenia el funcionament d'alguns aspectes clau pel correcte funcionament. Per aquest motiu, algunes trames no funcionen.

Durant tot el desenvolupament de la fase s'ha anat fent prova i error, de manera que s'anaven provant els apartats independentment per avançar de manera segura.

La comprovació d'errors ha sigut un punt clau que ha sigut de molta ajuda per detectar possibles falles en el programa i poder solventar-les. També s'han tingut en compte molts dels errors que poden donar-se. En canvi, no es comprova el correcte enviament de les trames amb el checksum.

### 3.3 FASE 3



Com ja hem comentat anteriorment, la fase 2, ens va generar més problemes dels que esperàvem, això ha fet que ens requerís més temps d'implementació, cosa que ens ha portat a dedicar més temps a l'enviament de trames per establir connexions, però un cop vam tenir la fase 2 acabada, poder gestionar i implementar la fase 3, ha estat molt més gratificant i fàcil.

Hem dividit la feina, per a cada una de les comandes noves que s'havia d'implementar o modificar per afegir, l'enviament de trames, així doncs, hem dedicat un temps per a cada una d'aquestes comandes. En funció de la dificultat de cada trama s'ha requerit més o menys temps d'implementació.

A més a més, constantment hem estat fent testing per poder comprovar que tot funcionava correctament i passant valgrinds, per evitar pèrdues de memòria o tancament incorrecta de fitxers, ja que en treballar amb tants fitxers, i memòria dinàmica és fàcil despistar-se i perdre memòria pel camí.

Per altra banda, hem estat revisant que es complissin tots els requisits i que s'enviessin els missatges corresponents a cada cas, a més, d'afegir comentaris extres de debug per poder saber en tot moment com està l'enviament de les trames entre connexions.

Ja per acabar com sempre hem avançat la memòria escrita, per poder tenir documentat tots els passos i en què ens anem trobant en cada moment.

## 3.4 FASE 4

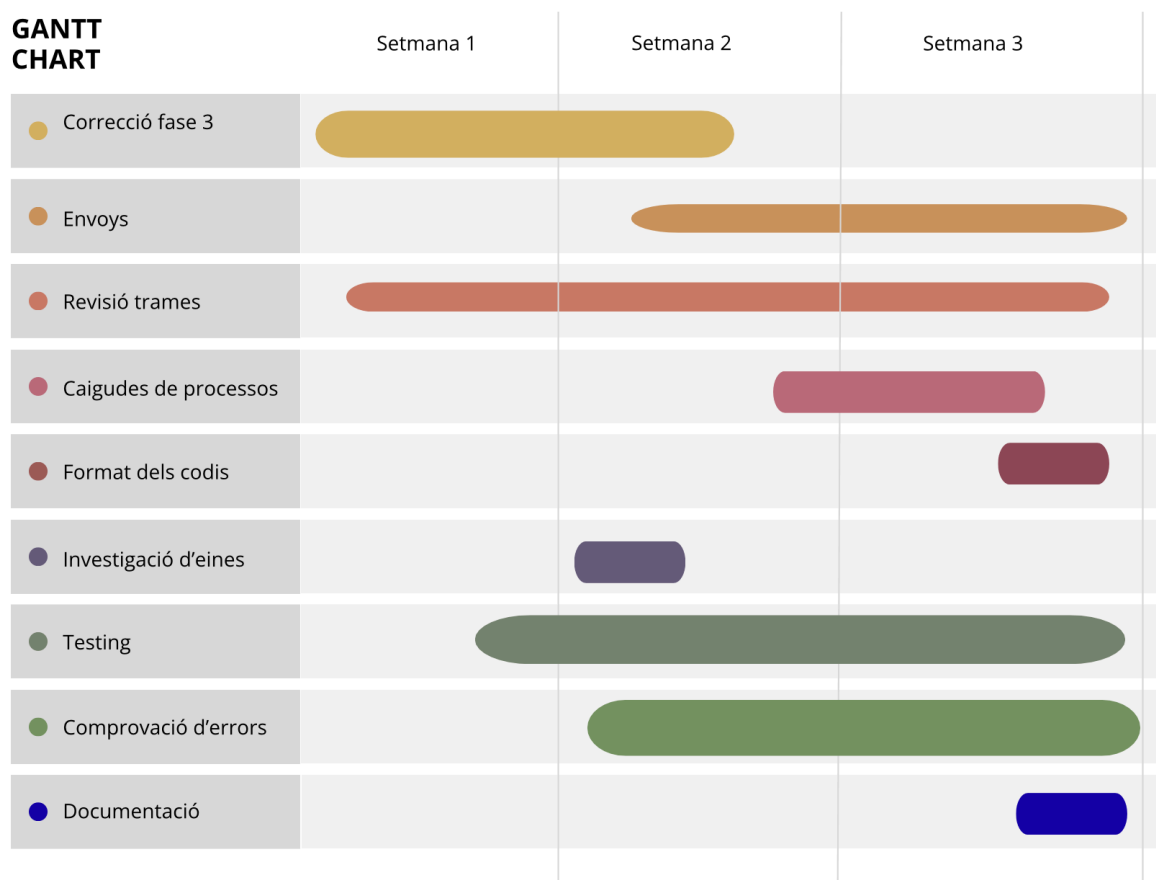
Primer de tot, com s'ha fet a la resta de fases anteriors, s'ha arreglat problemes reflectits al feedback de les entregues. Fent això, s'aconsegueix tenir una bona base amb la qual poder començar a treballar en els requisits de la fase actual.

En aquesta fase, el principal tema era poder implementar els envoyos, així doncs fer ús dels forks i pipes. Primer de tot poder establir comunicacions amb més d'un envoy, sempre comprovant l'estat i disponibilitat, i després executant les comandes. A més a més, s'anava establint la comunicació entre els pares i els fills amb pipes a mesura que es feien les comandes que requerien aquests recursos.

També hem destinat temps a poder rectificar les trames, fer ús de totes i intentar que tots els missatges de debug que surten per pantalla siguin útils pel correcte desenvolupament de la pràctica. Així doncs, també aplicant l'ús del mutex per pantalla.

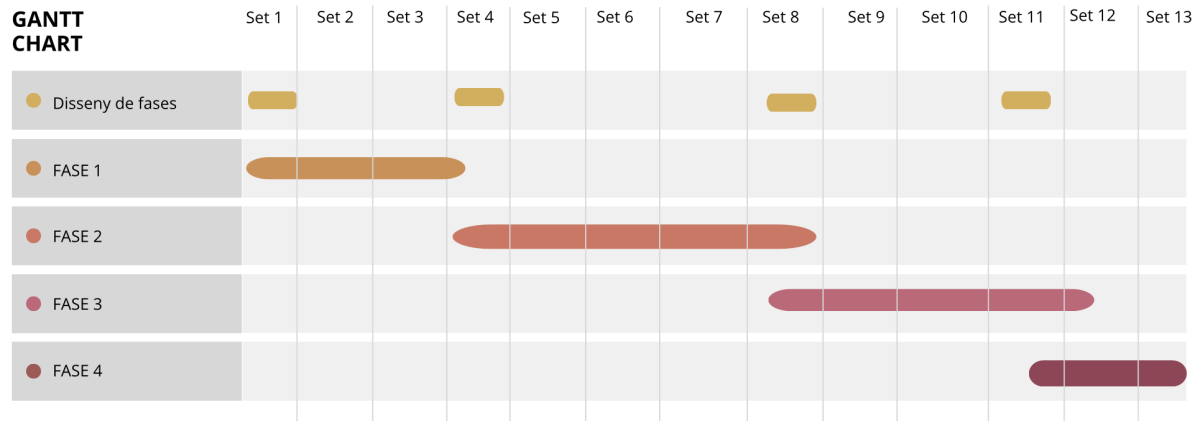
Finalment, s'ha revisat que es complissin tots els requisits de la pràctica, a més de la revisió final, de quan cauen els processos evitant pèrdues i fitxers oberts. A part, d'una dedicació a revisar les estructures dels codis i els formats, per seguir els guions penjats a l'estudy.

Pel que fa a la memòria, s'ha redactat i revisat el final, ja que aquesta vegada sí que era l'entrega definitiva. Tot i així, haver-la redactat per fases i ha facilitat la seva escriptura.





Finalment podem dir que hem pogut realitzar la pràctica amb aproximadament 13 setmanes, on la segona i tercera fase ens han requerit de molt més temps, ja que hem tingut més entrebancs però que tot i així, tot i les hores dedicades, especialment aquest desembre i Nadal, hem pogut acabar la pràctica dins del termini i amb un bon resultat.



## 4.CONCLUSIONS I MILLORES

La pràctica ha suposat una posada en pràctica de tots els recursos i eines vistes a classe en un entorn més realista i possible en el qual ens podem trobar dins del món dels sistemes operatius. Sobretot amb la gestió dels processos, fills, comunicacions entre processos i sincronització en entorns concurrents.

Així doncs, ens ha permès aplicar forks, pipes, signals, threads mutex i sockets, tot respectant les restriccions imposades, igual que les llibreries o funcions que no es podien fer servir.

A més a més, hem après a treballar mitjançant la comunicació per trames, amb els protocols establerts avui en dia, els quals són molt útils per comunicar processos i que ens han servit molt.

El resultat final obtingut és un sistema robust, modular i concurrent que compleix tots els requisits de les quatre fases i que estem contentes amb el que hem aconseguit:

En primer lloc, una primera correcta gestió de la configuració i inventari persistent.

Una comunicació entre regnes mitjançant les trames esmentades anteriorment, i el treball de reenviament hop-by-hop, cosa que és més difícil d'implementar que no les directament connectades, però que un cop entès el mecanisme del desplegament de les xarxes té una coherència i un sentit.

El suport per aliances, comerços i missions sortints, com la implementació dels envois com a processos independents que permeten múltiples missions simultànies sense bloquejar el Maester principal.

Una de les dificultats més importants, ha estat l'escolta activa permanent amb threads perquè el maester ha d'estar constantment pendent de moltes coses, o l'ús dels forks, pels envois combinat amb pipes i mutex per a l'exclusió mútua i la comunicació. Cosa que ens ha fet adonar que els sistemes operatius han de ser robustos, eficients i ben estructurats, ja que són complexes i contenen moltes coses a tenir en compte, perquè un sol error pot fer caure tot un sistema.

Pel que fa a possibles propostes de millora del sistema podríem afegir un timeout per a qualsevol enviament de trames, ja que si cau un procés, l'altre no estigui pendent d'espera. Igual que la gestió d'errors, que es podrien intentar tractar millor o de reconexió en cas de desconexió d'un aliat.

I com sempre, una millor optimització de la memòria, ja que hem intentat procurar sempre ocupar el mínim espai possible, però a vegades es podria fer encara més.

En definitiva, aquesta pràctica ha consolidat el coneixement teòric sobre concurrència i comunicació en sistemes operatius, mostrant com els mecanismes de baix nivell del SO poden construir aplicacions distribuïdes complexes i fiables.

## 5. NOMS DELS PROCESSOS

En aquesta pràctica es poden veure moltes referències a la serie “Juego de Tronos”. Principalment pels regnes independents que es comuniquen dins la xarxa. Aquests regnes presentats tenen els mateixos noms que es poden veure a la serie.

Un altre simil que es pot extreure, son els missatges que s’envien entre els diferents regnes. Tal com es fa a la pràctica, les trames s’envien per sockets i simulen el comportament de com enviaven a la serie, enganxant missatges a les potes dels corbs i anaven volant als regnes.

D'altra banda, com es fa a la serie, la forma d'identificar que un missatge és realment del regne corresponent és amb el segell de la casa reial, el mateix que es fa a la pràctica, enviant juntament la foto del segell amb la petició d'aliança.

Finalment, a la primera fase es pot veure com com el títol es Valar Compilis, similar a la frase de la serie Valar Morghulis, que significa que tots els homes han de morir. En aquest cas significa que tot ha de compilar.

## 6.BIBLIOGRAFIA

IBM Corporation. (s. f.). *strcasecmp()* — *Compare Strings without Case Sensitivity*.

Recuperat el [15 d'octubre], de

<https://www.ibm.com/docs/es/i/7.6.0?topic=functions-strcasecmp-compare-strings-without-case-sensitivity>

IBM Corporation. (s. f.). *strtok()* – *Tokenizar serie*. Recuperado el [18 d'octubre], de

<https://www.ibm.com/docs/es/i/7.6.0?topic=functions-strtok-tokenize-string>

IBM Corporation. (s. f.). *strrchr()*- *Localizar última aparición de carácter en serie*. Recuperat el [20 d'octubre], de

<https://www.ibm.com/docs/es/i/7.5.0?topic=functions-strchr-locate-last-occurrence-character-in-string>

IBM Corporation. (s. f.). *strdup()*- *reserva espacio de almacenamiento para una copia de serie con malloc*. Recuperat el [23 de novembre], de

<https://www.ibm.com/docs/es/i/7.6.0?topic=functions-strdup-duplicate-string>

IBM Corporation. (s. f.). *malloc, free, realloc, calloc, mallopt, mallinfo, mallinfo\_heap, alloca, valloc o posix\_memalign Subroutine*. Recuperat el [27 de desembre], de

<https://www.ibm.com/docs/es/aix/7.2.0?topic=m-malloc-free-realloc-calloc-mallopt-mallinfo-mallinfo-heap-alloca-valloc-posix-memalign-subroutine>